OOP SOLUTIONS

Mobile Shop

Developer and Setup Manual

What the software needs, how to set it up on a PC or a server, how the code is laid out, and how to change it safely.

OOP Solutions · mobileshop.oopsolutions.com · 2026-11-08

Contents

1. What the system is

2. What you need

3. Set up a development PC

4. Run it

5. Configuration

6. The code

7. Database and migrations

8. Tests

9. Rules every change must follow

10. Making a change

11. Production setup on a VPS

12. The Android app and the Windows PC version

13. Troubleshooting

1. What the system is

Mobile Shop is a mobile-shop management SaaS for Pakistani shops. It handles new and used phones (with ID checks and a full per-IMEI history), tablets and smart watches, accessories, purchases and sales, credit khata (udhaar), len-den, repairs and outside repair labs, installments, expenses, FBR tax, reports, the Shop Audit, staff and attendance, bulk Excel, barcodes and labels, subscription plans and a super-admin console. The interface works in three languages: English, Urdu (Nastaliq, right to left) and Roman Urdu.

This manual is for two kinds of reader: a developer who is new to the code, and a technical person who must install the software. It covers a development PC, the offline Windows PC version for one shop, and a production Linux server.

The parts at a glance

Part	Technology	Where in the repo
API server	Python 3.12, FastAPI, SQLAlchemy 2 (typed), Alembic, Pydantic v2	`backend/`
Database	PostgreSQL 16 in production; SQLite on a development PC and the offline Windows version	`backend/alembic/` (migrations)
Web app	React 18, Vite 5, TypeScript, Tailwind 3, react-i18next, TanStack Query, Zustand; installable PWA	`frontend/`
Android app	Capacitor 6 wrapping the same React build (app id `pk.mobileshop.app`)	`frontend/android/`
Web server in production	Caddy 2.8 serving the built app and passing `/api` and `/health` to the backend	`deploy/Caddyfile`, `deploy/web.Dockerfile`
HTTPS in production	An existing Traefik on the server (the stack opens no ports itself)	`deploy/docker-compose.yml` labels
Packaging	Docker images built by GitHub Actions and pushed to `ghcr.io`	`.github/workflows/`
Offline PC version	The same backend, bundled with PyInstaller, with a tray launcher and an Inno Setup installer	`local/`, `backend/app/local.py`

How a request flows

A shopkeeper's phone or PC opens the web app (or the Android app). The app calls the API under `/api/v1/...` with two headers: `Authorization: Bearer <access token>` and `X-Shop-Id: <shop uuid>`. In production, Traefik ends HTTPS and hands the request to the `web` container (Caddy), which passes `/api/*` and `/health` to the `backend` container (uvicorn). The backend works out the user and the active shop (`backend/app/core/tenancy.py`), checks the user's role capability and the shop's plan, runs the business logic in the module's `service.py` , writes to PostgreSQL in one transaction and returns JSON. Money is sent as strings with two decimals (for example `"25000.00"`).

Multi-tenancy in one paragraph

The hierarchy is `organisations` → `shops` → everything else. Every table that belongs to a shop has a `shop_id` column (the `ShopScoped` mixin in `backend/app/db/base.py`), and every service filters by the current shop explicitly. A row from another shop is reported as "not found", never "forbidden". There is no PostgreSQL row-level security; isolation is enforced in code and tested with two shops in `backend/tests/test_tenancy.py` and many module tests. The offline Windows version runs the same code with a single shop.

The two append-only logs

Two tables are the heart of the system and are never edited:

- `ledger_entries` : every movement of money, written as a balanced double-entry group by the posting service in `backend/app/modules/ledger/service.py` . Party balances, cash in hand, statements and profit are all derived from it.
- `device_events` : every event in a phone's life (purchase, sale, loan out, return, repair in and out, verification, price change, reclassification). The device's status is a projection checked by the state machine in `backend/app/modules/devices/state.py` ; the history screen is built from these rows.

Corrections are new reversing entries, never updates or deletes.

Background work

There is no separate worker. A small in-process clock (`backend/app/modules/notifications/scheduler.py`) ticks every minute inside the API: it sends the night report and reminders at each shop's time and dispatches queued WhatsApp and email messages. Backups in production are a cron job on the server (`deploy/backup.sh`).

2. What you need

Software

Tool	Version	Needed for
Python	3.12 exactly (`requires-python = ">=3.12,<3.13"` in `backend/pyproject.toml`)	the backend, tests, the Windows build
Node.js	22 (what CI and the web image use)	the frontend, Playwright, the screenshot and manual scripts
Git	any recent version (Git for Windows includes Git Bash)	the code
Docker with Compose v2	recent	optional on a PC (PostgreSQL, the production stack); required on the server
Inno Setup 6	6.x, with `iscc` on PATH	only to make the Windows `Setup.exe`
JDK 17 and the Android SDK (Android Studio)	JDK 17	only to build the Android app on your own PC; CI builds it without them

Check what is installed with `py -3.12 --version` (Windows) or `python3.12 --version` (Linux and macOS), `node --version` and `git --version`.

Accounts and services

What	Required?	Used for
GitHub account with access to the repository	yes	code, CI, the image registry (`ghcr.io`), the Android and Windows builds
A Linux VPS (Ubuntu or Debian, 2 GB RAM or more)	for production	runs the Docker stack
A domain or sub-domain	for production	the app's address; HTTPS is required for the camera, the PWA install and passkeys

What	Required?	Used for
Meta WhatsApp Business (Cloud API)	optional	automatic WhatsApp messages and OTP codes; without it messages wait in More → Messages with a tap-to-send link
An SMTP mailbox (for example Gmail with an app password)	optional	emails
An Anthropic API key	optional	the AI assistant and reading photos (CNIC, box labels)
A Google Drive account and rclone	optional	an encrypted off-server copy of the nightly backup

3. Set up a development PC

Paths below are relative to the repository root. On Windows the commands are for PowerShell unless noted.

Windows: get the code

1. Install Python 3.12, Node 22 and Git for Windows.
2. Clone the repository: `git clone https://github.com/AbdurRehmanAfzal/mobile-shop-software.git`
3. Go in: `cd mobile-shop-software`
4. Switch to the working branch: `git checkout develop` (all work happens on `develop`; `main` is for releases).
5. Check the commit author is set for this repository: `git config user.name` and `git config user.email` (see chapter 10 for the rule).

Windows: always set PYTHONUTF8

Windows Python writes console output in the cp1252 code page by default. Urdu text in seeds, error messages and receipts then crashes with `UnicodeEncodeError`. Set the variable in every terminal before running Python:

- PowerShell, this terminal only: `$env:PYTHONUTF8 = "1"`
- PowerShell, permanently for new terminals: `setx PYTHONUTF8 1`
- cmd.exe: `set PYTHONUTF8=1`
- Git Bash: `export PYTHONUTF8=1`

The Docker image, CI and the Windows build already set it.

Windows: backend

1. Go to the backend: `cd backend`
2. Create the virtual environment: `py -3.12 -m venv .venv`
3. Install the app and the developer tools (pytest, hypothesis, ruff, httpx, zxing-cpp): `.venv\Scripts\pip install -e ".[dev]"`
4. Optional, to work against PostgreSQL as well: `.venv\Scripts\pip install -e ".[dev,postgres]"`
5. Create your settings file from the example: `copy .env.example .env`
6. Edit `backend/.env` for local work (see below).
7. Set `$env:PYTHONUTF8 = "1"` in this terminal.
8. Create or update the SQLite database `backend/mobile_shop_v2.db`: `.venv\Scripts\alembic upgrade head`

Windows: the local backend/.env

`backend/.env` is ignored by git; never commit it. Every value is optional and the defaults suit a development PC (`backend/app/core/config.py`). For local work, add these lines on top of the example:

Line	Why
`PUBLIC_SIGNUP=true`	lets you and the Playwright tests sign up throwaway shops (production keeps it `false`)
`SUPERUSER_PHONES=<your-test-number>,03990000099`	your own test login gets More → Super admin; `03990000099` is the super user the Playwright specs use
`OTP_DEV_ECHO=true`	returns the OTP code in the API response; needed by `e2e/otp.spec.ts` ; never on a server
`RENEWAL_PRIVATE_KEY=<dev private key>` and `RENEWAL_PUBLIC_KEY=<dev public key>`	needed for license and renewal keys and `e2e/provider-console.spec.ts` ; make a pair with `.venv\Scripts\python -m app.modules.plan.renewal keygen`

> **Important:** never put a real customer's number, the production owner's login, production passwords or production keys in `.env` examples, tests, docs or commits. The development key pair is for the development PC only.

Windows: frontend

1. Open a second terminal and go to the frontend: `cd frontend`
2. Install the packages: `npm install`
3. Optional: `copy .env.example .env` . The only key is `VITE_API_URL` ; leave it empty so the Vite dev server proxies `/api` and `/health` to `http://localhost:8000` .
4. Install the Playwright browser once (for the end-to-end tests and the screenshot scripts): `npx playwright install chromium`

Linux and macOS notes

The steps are the same with Unix paths:

- Create the environment: `python3.12 -m venv .venv`
- Install: `.venv/bin/pip install -e ".[dev]"`
- Copy the settings: `cp .env.example .env`
- Set the encoding (harmless on Linux): `export PYTHONUTF8=1`
- Migrate: `.venv/bin/alembic upgrade head`

- Run tools as `.venv/bin/pytest`, `.venv/bin/ruff`, `.venv/bin/uvicorn`, or activate the environment with `source .venv/bin/activate`.

In Git Bash on Windows the scripts are under `.venv/Scripts/` instead of `.venv/bin/`.

Optional: a local PostgreSQL

SQLite is enough for daily work, but production is PostgreSQL 16, so check database-sensitive changes there too. A throwaway server with Docker:

1. Start it: `docker run -d --name ms-pg -e POSTGRES_USER=ms -e POSTGRES_PASSWORD=<local-password> -e POSTGRES_DB=ms_test -p 5432:5432 postgres:16-alpine`
2. Install the driver: `.venv\Scripts\pip install -e ".[dev,postgres]"`
3. Point the backend at it in `backend/.env`: `DATABASE_URL=postgresql+psycopg://ms:<local-password>@localhost:5432/ms_test`

4. Run it

The backend

1. In `backend/`, with `PYTHONUTF8=1` set: `.venv\Scripts\uvicorn app.main:app --reload --reload-dir app --port 8000`
2. Check it answers: open `http://localhost:8000/health`, which returns `{"status": "ok", "env": "dev"}`.
3. The interactive API documentation is at `http://localhost:8000/docs` (hidden when `APP_ENV=prod`).

On start, the app seeds the shared phone catalogue (brands, models, storages, colours) and the built-in expense categories. This is idempotent and safe on every start. The scheduler also starts unless `SCHEDULER_ENABLED=false`.

> **Tip:** after adding a new route, restart uvicorn. The reloader on Windows has missed new routes, and an orphan worker has kept port 8000 with old code; see chapter 13.

The frontend

1. In `frontend/`: `npm run dev`
2. Open `http://localhost:5173`. The dev server proxies `/api` and `/health` to port 8000.

The dev-server scripts in `frontend/package.json`:

Script	Address	Use
`npm run dev`	`http://localhost:5173`	normal work and Playwright
`npm run dev:lan`	`http://<PC-IP>:5173`	a phone on the same Wi-Fi (plain http)
`npm run dev:phone`	`https://<PC-IP>:5174`	a phone with the live camera (IMEI and barcode scanning); self-signed certificate, accept the warning once
`npm run preview`	`http://localhost:4173`	check a production build locally

A phone on the same Wi-Fi

1. Find the PC's address: `ipconfig` (the IPv4 address of the Wi-Fi adapter).
2. Start `npm run dev:lan`, or `npm run dev:phone` when you need the in-page camera.
3. On the phone open `http://<PC-IP>:5173` (or `https://<PC-IP>:5174`).
4. When Windows asks, allow Node on Private networks. If the prompt was dismissed, an administrator PowerShell can open the ports: `New-NetFirewallRule -DisplayName "Vite dev`

(mobileshop)" -Direction Inbound -Protocol TCP -LocalPort 5173,5174 -Action Allow -Profile Private

Only the Vite port needs to be reachable; Vite proxies the API to `localhost:8000`.

> **Note:** a phone browser opens the camera inside the page only on https or localhost. On the plain http link Android opens the separate camera app and Chrome may reload the page, and `crypto.randomUUID` and the clipboard are missing. Use `dev:phone` for scanning.

The first login

- With `PUBLIC_SIGNUP=true`, open `http://localhost:5173/signup` and create a shop.
- Or use the same command as on a server, which asks for the password without echoing it: `.venv\Scripts\python -m app.cli create-owner`
- Use a number listed in `SUPERUSER_PHONES` if you want More → Super admin.

A demo shop full of sample data

`frontend/scripts/demo-shop.mjs` builds a shop with sample data in every module through the real API, so all the business rules apply (ledger, device events, stock batches, verification, price rules).

1. Start the backend on port 8000 (with `PUBLIC_SIGNUP=true`).
2. In `frontend/`: `node scripts/demo-shop.mjs`
3. Optional: choose the login with the variables `PHONE`, `PASS` and `SHOP`.

> **Important:** the script can target another server with `API=https://<domain>/api/v1`. Only do that with the product owner's permission; it creates a real shop there.

Other scripts in frontend/scripts

Script	What it does	Needs
`landing-assets.mjs`	screenshots and the sample video for the landing page, into `public/landing/`	backend on 8000, frontend on 5173 (ffmpeg optional for MP4)
`manual-shots.mjs`	user-manual screenshots in English, Urdu and Roman into `docs/manual/img/<lang>/`; `ONLY=name1,name2` retakes some	backend and frontend running
`manual-admin-shots.mjs`	super-admin and provider-console screenshots	backend and frontend; `SUPER_PHONE` and `SUPER_PASS` for a number in `SUPERUSER_PHONES`

Script	What it does	Needs
`manual-pdf.mjs`	turns `docs/manual/*.md` into print HTML and A4 PDFs; `node scripts/manual-pdf.mjs developer-en` builds just this book	Playwright Chromium

5. Configuration

All backend settings are read by `backend/app/core/config.py` (pydantic-settings) from environment variables or from a `.env` file in the backend's working folder. Names are not case-sensitive. On the server, `/opt/mobileshop/.env` feeds `deploy/docker-compose.yml`, which passes a fixed list of variables to the backend container. On the offline Windows version, `settings.env` in the data folder plays the same role.

> **Important:** this chapter names variables and shows placeholders only. Never copy a real secret, token, password or server address into documentation, tests or commits.

Backend settings

Name	Required	Example	What it does
`APP_ENV`	no (default `dev`)	`prod`	`dev`, `test` or `prod`. In `prod` the app refuses to start with a default or short `SECRET_KEY` and hides `/docs`.
`DATABASE_URL`	no (default SQLite)	`sqlite:///./mobile_shop_v2.db` or `postgresql+psycopg://<user>:<password>@<host>:5432/mobileshop`	The database. PostgreSQL needs the `postgres` extra (psycopg 3).
`SECRET_KEY`	yes in production	`<random 64 hex characters>`	Signs the login tokens. At least 32 characters in `prod`; generate with `openssl rand -hex 32`.
`ACCESS_TOKEN_MINUTES`	no (30)	`30`	Life of an access token.
`REFRESH_TOKEN_DAYS`	no (30)	`30`	Life of a refresh token.
`CORS_ORIGINS`	no	`["https://<your-domain>","https://localhost","capacitor://localhost"]`	Allowed browser origins, as a JSON list. The Android app needs `https://localhost` and `capacitor://localhost`.
`WEBAUTHN_RP_ID`	no (`localhost`)	`<your-domain>`	Passkey (fingerprint login) domain.

Name	Required	Example	What it does
WEBAUTHN_ORIGINS	no	https://<your-domain>	Comma-separated full origins allowed for passkeys.
PUBLIC_URL	no	https://<your-domain>	Address for links inside customer messages (the repair status page). Empty: the https entry of WEBAUTHN_ORIGINS; on localhost the link is left out.
UPLOAD_DIR	no (./uploads)	/data/uploads	Where CNIC photos, device photos, signatures and other uploads are stored.
DEFAULT_LOCALE	no (ur)	ur	Default language for new shops.
TIMEZONE	no (Asia/Karachi)	Asia/Karachi	The shop day and the scheduler's local time.
PUBLIC_SIGNUP	no (false)	false	When false, only the super admin opens shops (ADR-012). true on a development PC and the offline Windows version.
SUPERUSER_PHONES	yes in production	03XXXXXXXXX	Comma-separated login numbers of the product owner; they see More → Super admin and the provider console.
ADMIN_WHATSAPP	no	03XXXXXXXXX	Receives "fee proof submitted", "new shop request" and password-reset requests. Empty = the first super-user number.

Name	Required	Example	What it does
`WHATSAPP_TOKEN`	no	`<permanent system-user token>`	Meta WhatsApp Cloud API token. With `WHATSAPP_PHONE_ID`, messages go out by themselves; empty = sent by hand from More → Messages.
`WHATSAPP_PHONE_ID`	no	`<phone number id>`	The WhatsApp business number's id from Meta.
`WHATSAPP_API_VERSION`	no (`v21.0`)	`v21.0`	Graph API version. Not passed by the compose file.
`WHATSAPP_TEMPLATES`	no	`{"udhaar_reminder": {"name":"udhaar_reminder_ur","lang":"ur"}}`	JSON map from our message keys to approved Meta templates (one body parameter each). Unmapped keys are sent as plain text.
`SCHEDULER_ENABLED`	no (`true`)	`true`	The in-process clock for night reports, reminders and the message queue.
`OTP_DEV_ECHO`	no (`false`)	`false`	Returns OTP codes in the API response, for local tests only. Never `true` in production.
`SIGNUP_OTP`	no (`false`)	`false`	Public signup must prove the phone number with a WhatsApp code.
`SMTP_HOST`	no	`smtp.gmail.com`	Mail server. Empty = emails stay queued in Messages.
`SMTP_PORT`	no (587)	`587`	Mail server port.
`SMTP_USER`	no	`<mailbox>`	Mail login.
`SMTP_PASSWORD`	no	`<app password>`	Mail password (for Gmail, an app password).
`SMTP_FROM`	no	`<sender address>`	The From address.

Name	Required	Example	What it does
SMTP_STARTTLS	no (true)	true	Use STARTTLS.
ANTHROPIC_API_KEY	no	<your-api-key>	Switches on the AI assistant and photo reading. Empty = the feature says it is not switched on.
AI_MODEL	no (claude-opus-5-5)	claude-opus-5-5	Model for the assistant chat.
AI_VISION_MODEL	no (claude-haiku-5-5)	claude-haiku-5-5	Cheaper model for reading photos.
AI_DAILY_LIMIT	no (200)	200	AI calls per shop per day (chat and photos together).
RENEWAL_PRIVATE_KEY	only on the SaaS server	<private key from keygen>	Signs license and offline renewal keys. Keep it only on the production server.
RENEWAL_PUBLIC_KEY	no (a built-in default)	<public key from keygen>	Checks those keys; every install, including the offline Windows version, carries it.
WEB_DIR	no	C:\Program Files\Mobile Shop\web	When set, the backend also serves the built web app from this folder (used by the offline Windows version).
LOCAL_MODE	no (false)	true	The offline one-PC install: its organisation is plan chain with no expiry.
SAAS_URL	no	https://<your-domain>	Only with LOCAL_MODE : the offline install reports a heartbeat (version, plan end, keys used) to this server when it has internet. Empty = never.
APP_VERSION	no (2.0)	2.1.0	Version sent in that heartbeat.

Server .env (deploy/.env.example)

These are the keys of `/opt/mobileshop/.env` on the server. `setup-vps.sh` creates the file from `deploy/.env.example` and fills in fresh database and secret passwords.

Name	Required	Example	What it does
`APP_DOMAIN`	yes	`app.example.com`	The app's address. Used for the Traefik route, CORS and passkeys. Its DNS A record points to the server.
`POSTGRES_DB`	no (`mobileshop`)	`mobileshop`	Database name.
`POSTGRES_USER`	no (`mobileshop`)	`mobileshop`	Database user.
`POSTGRES_PASSWORD`	yes	`<openssl rand -hex 24>`	Database password; compose refuses to start without it.
`SECRET_KEY`	yes	`<openssl rand -hex 32>`	As above; compose refuses to start without it.
`WEB_WORKERS`	no (2)	`2`	Number of uvicorn worker processes.
`PUBLIC_SIGNUP`	no (`false`)	`false`	Keep `false` in production.
`SUPERUSER_PHONES`	yes	`03XXXXXXXXX`	The product owner's login number(s).
`IMAGE_PREFIX`	no	`ghcr.io/<owner>/mobile-shop`	Where the images come from; `-backend` and `-web` are appended.
`IMAGE_TAG`	set by `deploy.sh`	`<git sha>`	Which image version runs.
`WHATSAPP_TOKEN` , `WHATSAPP_PHONE_ID` , `WHATSAPP_TEMPLATES`	no	see above	Automatic WhatsApp.
`RCLONE_REMOTE`	no	`gdrive-safe:`	Read by `backup.sh` only: copies each night's backup to this rclone remote.

Name	Required	Example	What it does
`SIGNUP_OTP`, `OTP_DEV_ECHO`	no	`false`, `false`	As above.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`, `SMTP_STARTTLS`	no	see above	Email.
`ANTHROPIC_API_KEY`, `AI_MODEL`, `AI_VISION_MODEL`, `AI_DAILY_LIMIT`	no	see above	AI.
`ADMIN_WHATSAPP`	no	`03XXXXXXXXX`	As above.
`RENEWAL_PRIVATE_KEY`, `RENEWAL_PUBLIC_KEY`	yes for license keys	from keygen	As above.

The compose file sets `APP_ENV=prod`, `DATABASE_URL` (built from the `POSTGRES_` values, host `db`), `CORS_ORIGINS` (`https://APP_DOMAIN` plus the two Capacitor origins), `UPLOAD_DIR=/data/uploads`, `WEBAUTHN_RP_ID`, `WEBAUTHN_ORIGINS` and `PUBLIC_URL` itself.

> **Note:** the compose file passes only the variables listed in its `backend.environment` block. `SCHEDULER_ENABLED`, `DEFAULT_LOCALE`, `TIMEZONE`, `WHATSAPP_API_VERSION` and `SAAS_URL` are not in that block, so putting them in `/opt/mobileshop/.env` has no effect until a line is added to `deploy/docker-compose.yml`. Any new backend variable must be added there too, and to `docs/29-OWNER-PENDING.md`.

Frontend

Name	Required	Example	What it does
`VITE_API_URL`	no	empty, or `https://<your-domain>`	Base address of the API, fixed at build time. Empty = same origin (the dev proxy, the web image and the offline version). The Android build sets it to the live server.

Offline Windows version

Name	Where	What it does
`MOBILESHOP_DATA`	environment	Data folder. Default `%LOCALAPPDATA%\MobileShop`.
`MOBILESHOP_PORT`	environment	Port, default `8787`.
`MOBILESHOP_HOST`	`settings.env` or environment	`0.0.0.0` opens the app to phones on the shop's Wi-Fi; default `127.0.0.1`.
`MOBILESHOP_LOG`	environment	Write `mobileshop.log` also when running from source.

On first run the launcher writes `settings.env` with `APP_ENV=prod`, a random `SECRET_KEY`, the SQLite path, the upload folder, `PUBLIC_SIGNUP=true`, `LOCAL_MODE=true`, `SCHEDULER_ENABLED=true` and matching CORS and passkey origins. WhatsApp, email and AI keys use the same names as the server and can be added there.

GitHub secrets and variables

Set them in the repository under Settings → Secrets and variables → Actions.

Name	Kind	What it is for
`VPS_HOST`	secret	The server's address for the deploy job.
`VPS_USER`	secret	The restricted SSH user, `deploy`.
`VPS_SSH_KEY`	secret	The private SSH key whose public half is in `/home/deploy/.ssh/authorized_keys`.
`APP_URL`	variable	`https://<your-domain>`: the Android build's API address and the final health check after a deploy.
`ANDROID_KEYSTORE_BASE64`	secret	The release keystore file, base64-encoded on one line.
`ANDROID_KEYSTORE_PASSWORD`	secret	Its password.
`ANDROID_KEY_ALIAS`	secret	The key alias, for example `mobileshop`.

The deploy job also uses a GitHub environment named `production`; the image push uses the built-in `GITHUB_TOKEN`.

6. The code

Repository layout

Path	What it holds
`CLAUDE.md`	Project context and the hard rules (read it first).
`README.md`	Documentation index.
`docs/`	Living documentation: requirements, architecture, data model, progress, session log, decisions (ADRs), design system, deployment, plans for each programme; `docs/guides/` and `docs/manual/` for readers; `docs/archive/` for the old v1 docs.
`backend/`	The FastAPI application, migrations and tests.
`frontend/`	The React app, the Android project, Playwright specs and helper scripts.
`deploy/`	Compose file, Caddyfile, web image, server scripts and the CI smoke test.
`local/`	The offline Windows build: `build.ps1`, `mobileshop.spec` (PyInstaller), `installer.iss` (Inno Setup).
`.github/workflows/`	`ci.yml`, `deploy.yml`, `android.yml`, `windows.yml`.
`.claude/skills/`	Project skills for AI-assisted sessions: `kickoff`, `handoff`, `pk-mobile-domain`.
`legacy/`	The v1 prototype, for reference only; never extend it.

Backend structure

Path	What it holds
`backend/app/main.py`	`create_app()`: the production-secret check, CORS, error handlers, `/health`, the `autoprint` middleware for print pages, every module's router, and the optional web-app mount (`WEB_DIR`).
`backend/app/cli.py`	`python -m app.cli create-owner`: the first login on a server.
`backend/app/local.py`	The offline Windows launcher.
`backend/app/core/config.py`	Settings.

Path	What it holds
backend/app/core/tenancy.py	The current user and shop (Ctx), the DB session dependency, require_cap(...) and the plan gate.
backend/app/core/permissions.py	Roles and capabilities.
backend/app/core/errors.py	AppError and its subclasses with a code and English and Urdu messages.
backend/app/core/security.py , ratelimit.py , idempotency.py	Tokens and bcrypt, rate limits, the Idempotency-Key handling for offline documents.
backend/app/core/pk.py , clock.py , periods.py	Pakistani helpers (phone numbers, CNIC, IMEI Luhn), the shop clock, report periods.
backend/app/db/base.py	Base , the UUIDPk , Timestamped and ShopScoped mixins, Decimal mapped to Numeric(14,2) .
backend/app/db/models.py	Imports every module's models, so Alembic and the metadata see all tables.
backend/app/db/session.py	Engine, SessionLocal , get_db .
backend/app/seeds/catalogue.py	The shared brands, models, storages and colours.

Each domain module under backend/app/modules/<name>/ usually has models.py , schemas.py , service.py and router.py . Routers stay thin (dependencies, then one service call); every rule lives in service.py , so tests and other services can reuse it.

Module	Responsibility
auth	Login, tokens, PIN switch, shops, staff, the business profile.
admin	Super admin: organisations, plans, fees, the provider console, license keys.
plan	Subscription stages, reminders, the clock guard, offline renewal keys.
catalogue	Brands, models, storages, colours, the per-shop model library.
parties	Customers, suppliers, dealers, technicians (repair labs): one khata per person.
ledger	The posting service, account keys and statements.
devices	Phones, tablets and watches: events, state machine, device types and spec keys, pricing, ageing.

Module	Responsibility
`verification`	ID checks (CNIC, eGadget, PTA) and the signed seller declaration.
`purchases` , `sales` , `returns` , `exchanges`	The buying, selling, return and exchange documents.
`payments`	Money in and out, allocation against bills, len-den (`purpose = 'lending'`) and its due dates.
`paymode`	Which money account (cash drawer, bank, wallet) a payment lands in, by way of paying.
`loans`	Phones lent to and borrowed from other shops (amanat).
`installments`	Qist plans on top of an invoice.
`invoices`	One read-only list of every bill (sales, purchases, returns, repair jobs) to search and filter.
`repairs`	Repair jobs, repair labs and dispatches.
`accessories`	Counted items with purchase batches and price tiers.
`units`	Unit-wise money, derived: what one phone cost, sold for and still owes, per phone and per person.
`expenses`	Expenses and day close.
`reports` , `shopaudit` , `exports` , `bulk`	Reports, the read-only Shop Audit, CSV and Excel exports, bulk Excel import.
`barcodes` , `labels`	Barcodes assigned at purchase; label and repair-sticker print pages.
`notifications` , `otp`	The message queue, WhatsApp and email providers, the scheduler and its jobs; OTP codes.
`ai`	The assistant (read-only, shop-scoped tools) and photo reading.
`attendance` , `audit`	Staff attendance (hazri) and activity; the audit log.
`files` , `fingerprint` , `public`	Uploads; fingerprint login (app device keys and website passkeys); routes without login (plans and prices for the landing page, the "open a shop for me" request, the offline-install heartbeat).

Frontend structure

Path	What it holds
`frontend/src/main.tsx`	Providers: TanStack Query (with a persisted cache), i18n, router.
`frontend/src/app/router.tsx`	All routes.
`frontend/src/app/Shell.tsx`	Header, bottom tabs on a phone, side menu with F-keys on a laptop.
`frontend/src/components/`	Shared design-system pieces: `Bilingual.tsx` (`BiLabel` , `BiButton` , `BiInput` and friends), `ui.tsx` (page header, sheets, lists, money text, chips, sections), `Help.tsx` (the "?" sheets), pickers, scanners, `PlanReminders.tsx` , `ModuleGate.tsx` , `charts/` (in-house SVG).
`frontend/src/features/<module>/`	Pages per module: `counter` (home, hisaab, khata), `sale` , `purchase` , `stock` , `parties` , `payments` , `repairs` , `accessories` , `audit` , `reports` , `more` (settings, staff, bulk, profile), `admin` , `landing` and others.
`frontend/src/lib/api.ts`	Fetch wrapper: auth header, `X-Shop-Id` , token refresh, `ApiError` .
`frontend/src/lib/queries.ts`	Query hooks; keys start with the active shop id.
`frontend/src/lib/format.ts`	`formatMoney` and `withCur` (the only way to write money), dates.
`frontend/src/lib/offline.ts` , `outbox.ts`	Cached reads and the outbox of documents sent with an `Idempotency-Key` .
`frontend/src/lib/draft.ts` , `id.ts` , `scanner.ts` , `scan.ts`	Form drafts that survive a reload, ids without `crypto.randomUUID` , the hardware-scanner listener, camera scanning.
`frontend/src/i18n/index.ts`	i18next setup and `setLocale()` .
`frontend/src/i18n/locales/`	`en.json` , `ur.json` , `roman.json` , with the same keys in all three.
`frontend/src/index.css`	Design tokens (CSS variables) per look and theme.
`frontend/e2e/`	Playwright specs and `fixtures/` (barcode images, a fake camera stream).

Languages and i18n

- The UI shows one language at a time: Urdu mode is only Urdu (Nastaliq, right to left), English mode only English, Roman Urdu mode only Roman. Never stack English and Urdu labels.
- Every visible string is a key present in all three files. Labels take a key, never a literal sentence.
- Numbers are always Latin digits. Wrap them in the `.num` class or use the money components, because Nastaliq draws 0 as a dot.
- Use logical layout classes (`text-start` , `ps-` , `me-` , `start-`) so screens mirror correctly in Urdu.
- Server errors carry `message_en` and `message_ur` ; the app shows the one for the current language (Roman mode shows English).
- Printed documents and notifications follow `shops.default_locale` , overridable with `?lang=en` or `?lang=ur` .
- Never turn a key that is a namespace object into a string (that once wiped a whole group of labels); pick a new name instead.

Help terms

Every field, tab and card that needs explaining gets a "?" through the one `Help` component (ADR-043, `docs/33-HELP-EVERYWHERE-PLAN.md`). The texts live under `help.` in the locale files, each term with a title, an explanation and an example in all three languages. `frontend/src/components/Help.test.ts` scans the code for every term used and fails if one lacks a language or the example.

Design system

The approved design is "Dukaan counter" (`docs/11-DESIGN-SYSTEM.md`), with the default look "Modern" (ADR-011). Build new screens only from the existing components on the CSS tokens in `index.css` . Every action looks like a button and has an icon with words (lucide icons, never emoji); no default-blue buttons and no bare white cards; tap targets at least 44 px and main actions 56 to 64 px; at most three steps for daily jobs; never colour alone for meaning. Phone and laptop use the same components. Charts are in-house SVG, with no chart library.

7. Database and migrations

One schema, two databases

The same SQLAlchemy models run on PostgreSQL 16 (production and CI) and SQLite (a development PC, the offline Windows version, the unit tests). Alembic migrations live in `backend/alembic/versions/` (about fifty files, one chain). `backend/alembic/env.py` takes the database address from the app's settings and renders batch mode on SQLite (`render_as_batch`), because SQLite cannot alter most things in place. In production the backend container runs `alembic upgrade head` every time it starts (see `backend/Dockerfile`); the offline Windows launcher does the same with its bundled migrations.

Useful commands (in backend/)

Task	Command
Bring the database to the latest version	`.venv\Scripts\alembic upgrade head`
Show the current version	`.venv\Scripts\alembic current`
Show the chain	`.venv\Scripts\alembic history`
Draft a migration from model changes	`.venv\Scripts\alembic revision --autogenerate -m "short description"`
Undo one step (test the downgrade)	`.venv\Scripts\alembic downgrade -1`
On the server, check the version	`docker compose --env-file .env exec backend alembic current`

How to write a migration

1. Change the model in the module's `models.py` . A new table uses the `UUIDPk` , `ShopScoped` and `Timestamped` mixins, and money as `Mapped[Decimal]` (which maps to `Numeric(14,2)`). A new module's models must be imported in `backend/app/db/models.py` .
2. Bring your database to head first, then autogenerate.
3. Review the generated file; it is a draft. Put a one-line docstring on top that says what changed and why.
4. Keep `op.batch_alter_table(...)` blocks; they work on PostgreSQL too.
5. Name every constraint, and copy the exact names from `upgrade()` into `downgrade()` .
6. A new `NOT NULL` column on an existing table needs a `server_default` .
7. Remove anything autogenerate added that you did not intend.
8. Test up, down and up again, on SQLite and, before a release, on PostgreSQL.

Additive only

Migrations must be additive: add tables, add nullable columns or columns with defaults, add indexes. Never drop or rewrite data that shops have entered, and never edit an existing migration that has reached production. The Shop Audit, bulk Excel, barcodes, accessories, tablets and watches, len-den and repair-lab work were all delivered this way. Derived figures (balances, open len-den, unit-wise money, audit numbers) are computed, not stored.

> **Note:** SQLite's batch mode rebuilds the table and silently tolerates a wrong constraint name in a downgrade. PostgreSQL does not. A real bug of this kind (an FK named differently in upgrade and downgrade) was caught only by CI's PostgreSQL job.

`backend/tests/test_migrations.py` runs `alembic upgrade head` on an empty SQLite file and checks that it produces exactly the tables and columns of the models; a forgotten migration fails it.

8. Tests

What CI runs

Job	Steps
backend	`pip install -e ".[dev,postgres]"`, `ruff check app tests`, `pytest -q -p no:warnings` on SQLite, then `alembic upgrade head`, `alembic downgrade base`, `alembic upgrade head` on PostgreSQL 16
frontend	`npm ci --no-audit --no-fund`, `npm run typecheck`, `npm test`, `npm run build`
stack	builds the production images, runs db, backend and web together, then `deploy/ci/smoke.sh` (health, the app page, a deep link, signup, the seeded catalogue, a person, a phone, a report)

Playwright is not run in CI. Run the relevant specs on your PC after any UI change.

Backend (in backend/, PYTHONUTF8=1)

Task	Command
Lint, exactly as CI	`.venv\Scripts\ruff check app tests`
Auto-fix lint (import order and similar)	`.venv\Scripts\ruff check app tests --fix`
All tests	`.venv\Scripts\pytest -q`
One file	`.venv\Scripts\pytest -q tests/test_ledger.py`
Tests whose name matches	`.venv\Scripts\pytest -q -k "lending"`
Stop at the first failure, rerun the last failures	`.venv\Scripts\pytest -q -x --lf`

The tests use an in-memory SQLite database with a fresh schema, and `conftest.py` switches public signup on, so they do not need `backend/.env`. Useful fixtures in `backend/tests/conftest.py`: `client`, `db`, `owner` (a signed-up shop whose `.get/.post/.patch/.put/.delete` send the token and `X-Shop-Id`), `other_owner` (a second shop for leakage tests) and `cat` (the seeded catalogue). Property tests use hypothesis (for example, debits equal credits for any sequence of documents). IMEIs in tests must pass the Luhn check, for example `490154203237518` or `356938035643809`.

To run the same tests on PostgreSQL, point `TEST_DATABASE_URL` at a disposable database (all tables are dropped and recreated):

1. In PowerShell: `$env:TEST_DATABASE_URL = "postgresql+psycopg://ms:<local-password>@localhost:5432/ms_test"`
2. Run: `.venv\Scripts\pytest -q`
3. Clear it afterwards: `Remove-Item Env:TEST_DATABASE_URL`

To repeat CI's migration check on PostgreSQL, set `DATABASE_URL` the same way and run `alembic upgrade head`, `alembic downgrade base` and `alembic upgrade head`.

Frontend (in frontend/)

Task	Command
Type check (strict)	`npm run typecheck`
Unit tests (Vitest, jsdom)	`npm test`
Unit tests in watch mode	`npm run test:watch`
One test file	`npx vitest run src/lib/scan.test.ts`
Production build	`npm run build`

The unit tests include the money format, scanners, the theme, the bilingual primitives and `Help.test.ts` (every "?" term has all three languages and an example).

End-to-end (Playwright)

The specs in `frontend/e2e/` run in one project at phone size (a Pixel 7 profile); laptop checks set their own viewport inside the spec. They need:

- the backend on port 8000 with `PUBLIC_SIGNUP=true` (specs sign up throwaway shops with random numbers);
- `03990000099` in `SUPERUSER_PHONES` for the super-admin specs (`admin`, `admin-access`, `fees`, `plans`, `plan-reminders`, `modules`, `provider-console`, `landing`);
- `OTP_DEV_ECHO=true` for `otp.spec.ts`, and a renewal key pair for `provider-console.spec.ts`;
- **no** WhatsApp Cloud API or SMTP keys (`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `SMTP_HOST`): with them a spec would really send to its test numbers, so `reminders.spec.ts` skips itself. If your `.env` holds them, start the e2e backend from Git Bash with `WHATSAPP_TOKEN= WHATSAPP_PHONE_ID= SMTP_HOST=` in front of `uvicorn` (an empty variable wins over `.env`; in cmd, `set X=` removes it instead, so `.env` would win);
- the frontend on port 5173; Playwright starts `npm run dev` itself or reuses a running one.

Task	Command
All specs	`npm run e2e`
One file	`npx playwright test e2e/crud.spec.ts`

Task	Command
By test title	`npx playwright test -g "bill number"`
Watch the browser	`npx playwright test --headed`
Open the report after a failure	`npx playwright show-report`

`e2e/urdu.spec.ts` fails when English words leak into Urdu mode; run it after any screen change. `scan-live.spec.ts` feeds Chrome a fake camera from `e2e/fixtures/label-live.mjpeg`. Do not run two heavy Playwright suites on one PC at the same time. If `admin.spec.ts` times out in a full run, run it alone before investigating; it is a known flake under load.

Manual checks

Some things only a person can check: printing on a real 58 mm thermal printer, Urdu rendering on an Android phone, a real barcode scanner (`docs/23-BARCODES-PLAN.md`), live camera scanning on HTTPS. The full manual test plan is `docs/14-MANUAL-TEST-PLAN.md` ; the owner's open test list is in `docs/29-OWNER-PENDING.md` .

9. Rules every change must follow

These come from `CLAUDE.md` and the decisions in `docs/10-DECISIONS.md`. They are not optional.

Money

- Money is never a float. The database uses `Numeric(14,2)`, Python uses `Decimal`, built from strings, never from floats.
- Round with `money()` from `backend/app/modules/ledger/service.py` (two places, half up).
- The API sends money as strings with two decimals; tests compare strings.
- The currency is PKR. The frontend writes money only through `formatMoney` or `withCur` from `lib/format.ts`, never a literal "Rs.". Server totals are the authority; client arithmetic is only for live previews.

Tenancy

- Every tenant-owned table has `shop_id`, and every query filters by the current shop (`ctx.shop_id`).
- Load by id and shop; a row from another shop is "not found".
- Every id that arrives from the client (a party, a device, an account) must be checked to belong to the same shop.
- Shared rows (the seeded catalogue, built-in expense categories) have `shop_id` empty and are read together with the shop's own rows; shops may edit only their own.
- Every new endpoint gets a leakage test with `owner` and `other_owner`.

The ledger and device events

- The ledger is append-only. All money goes through `post(...)` in the ledger service, which refuses unbalanced groups. Never update or delete a `ledger_entries` row.
- Balances are derived, never typed in. A correction is a reversing group (`reverse(...)`) plus, if needed, a new posting.
- Balance sign: positive means the person owes the shop ("lena hai"), negative means the shop owes the person ("dena hai").
- Every device change is a new `device_events` row written through `apply_event(...)` in `backend/app/modules/devices/service.py`. Illegal transitions raise an error from the state machine. A phone bought again keeps its one `devices` row.
- Documents post their ledger entries and device events in one database transaction.

Errors

Raise the `AppError` subclasses from `backend/app/core/errors.py` (`NotFound`, `Conflict`, `ValidationFailed` and others) with a stable `code` and messages in English and Urdu. The response

shape is `{"error": {"code", "message_en", "message_ur", "details"}}`.

Language and UI

- One language at a time; never stack English and Urdu.
- Every string in `en.json`, `ur.json` and `roman.json`; no hard-coded text.
- Latin digits always. Shop words, not accounting words (udhaar, lena and dena, "stock daalein", "ID card check").
- Every "?" help term has a title, an explanation and an example in all three languages.
- New screens only from the design-system components; the UI must feel premium and modern (no emoji icons, no default-blue buttons, no bare white cards).

Programme-specific rules

Several areas have their own plan document and rules; read the plan before touching that code. Examples: the Shop Audit is read-only and every figure must be traceable (`docs/17-SHOP-AUDIT-PLAN.md`); bulk import only calls the existing services and previews before writing (`docs/18-BULK-EXCEL-PLAN.md`); a barcode is assigned in the purchase transaction and never changes (`docs/23-BARCODES-PLAN.md`); accessories stock moves through batches, oldest first (`docs/25-ACCESSORIES-PLAN.md`); `devices.category` is the only device type (`docs/27-TABLETS-WATCHES-PLAN.md`); len-den is a payment with `purpose = 'lending'` (`docs/32-LENDING-PLAN.md`); a lab's charge is booked on return and the repair sticker never prints the screen lock (`docs/34-REPAIR-LABS-PLAN.md`). `CLAUDE.md` lists all of them.

Tests and docs

- Business logic (ledger, balances, the device state machine, tax, numbering, tenancy) gets backend tests before the UI. Run the tests before committing.
- Stale docs are bugs. At the end of a piece of work, update `docs/06-PROGRESS.md`, append an entry to `docs/09-SESSION-LOG.md` (date, what was done, what was verified, what is next, open questions), add an ADR to `docs/10-DECISIONS.md` when a design decision was made, and keep `docs/04-DATA-MODEL.md` in step with the schema.
- Every new environment variable, server command or manual test step goes into `docs/29-OWNER-PENDING.md`, and every new backend variable into the compose file.

10. Making a change

Before you start

1. Read `docs/06-PROGRESS.md` (what is done, in progress and next).
2. Read the last entry of `docs/09-SESSION-LOG.md` (where the last session stopped and its open questions).
3. For business rules, read the relevant section of `docs/02-FUNCTIONAL-REQUIREMENTS.md` and the plan document for that area.
4. Only then read the code you need. Do not re-audit the whole repository, and do not extend `legacy/`.

With Claude Code, the project skills `/kickoff` and `/handoff` walk through the start and the end of a session.

A change from end to end

1. Model and a reviewed migration (chapter 7).
2. Schemas for input and output in `schemas.py`.
3. The rule in `service.py`: `Decimal`, shop checks, bilingual errors, ledger postings and device events through their services, one commit at the end.
4. The route in `router.py` with the right dependency (`Ctx`, or `require_cap(Cap.X)`). Put fixed paths before `/{id}` paths. A new module's router must be added in `backend/app/main.py`. Restart uvicorn.
5. Backend tests, including a tenancy test; `ruff` clean.
6. The frontend: a hook in `lib/queries.ts` or the generic hooks, the type in `lib/types.ts`, the screen from design-system components.
7. Keys in `en.json`, `ur.json` and `roman.json`; help terms if the field needs a "?".
8. A Playwright spec, then `e2e/urdu.spec.ts`; look at the Urdu screen.
9. Docs: data model, progress, session log, an ADR if a decision was made, the owner's pending list if something must be done on a server.

Branches

Branch	Use
`develop`	All work. CI runs on every push.
`main`	Releases only. Every push to `main` deploys to production.

Do not push unless the owner asks. A release means merging `develop` into `main` and pushing `main`, and only the owner decides when. Open a pull request from `develop` to `main`, let CI pass,

then merge; or, when the owner has said so, merge and push by hand:

1. `git checkout main`
2. `git pull`
3. `git merge develop`
4. `git push origin main`
5. `git checkout develop`

Commits

- The only commit author is `Abdur Rehman Afzal <abdurrehmanafzal786@gmail.com>`; the repository-local git config is already set. The project rule in `CLAUDE.md` forbids `Co-Authored-By` lines, "Generated with" lines or any AI attribution in commits, pull requests or files.
- Use short conventional messages that describe the visible result, for example `feat(repairs): ...`, `fix: ...`, `docs: session log - ...`, `ci: ...`, `style: ...`.
- Keep commits small, with tests green before each one.

What happens after a push

Event	Workflow	Result
Push to `develop`, or any pull request	`ci.yml`	backend, frontend and stack jobs
Push to `main`, or Actions → Deploy → Run workflow	`deploy.yml`	CI, then the Android APK (`android.yml`), then the backend and web images pushed to `ghcr.io` (tagged with the commit and `latest`), then SSH to the server and `deploy.sh <commit>`, then a check of `APP_URL/health`
Actions → Android → Run workflow	`android.yml`	an APK as a build artifact
Push to `main` touching `local/`, `backend/` or `frontend/`, or Actions → Windows → Run workflow	`windows.yml`	`MobileShop-Setup-<version>.exe` as a build artifact

The site is deployed even if the APK build fails; the new web image then has no APK, so `/app.apk` returns 404 until the next successful release.

11. Production setup on a VPS

How production is arranged

The production stack (`deploy/docker-compose.yml` , Compose project name `mobileshop`) has three containers:

Container	Image	Role
`db`	`postgres:16-alpine`	The database, volume `mobileshop_pgdata` , only on the stack's internal network.
`backend`	`<IMAGE_PREFIX>-backend:<tag>` from `backend/Dockerfile`	FastAPI; runs `alembic upgrade head` and then uvicorn on port 8000; uploads in volume `mobileshop_uploads` ; network alias `mobileshop-backend` .
`web`	`<IMAGE_PREFIX>-web:<tag>` from `deploy/web.Dockerfile`	Caddy on port 80 with the built React app (and `/app.apk` when the release built one); passes `/api` and `/health` to the backend.

The stack opens no ports on the server. HTTPS and routing come from a Traefik that already runs on the server: the `web` container joins the external Docker network `n8n_default` and carries Traefik labels for `Host(APP_DOMAIN)` on the entrypoints `websecure` (TLS with the certificate resolver `mytlschallenge`) and `web` (redirect to HTTPS). Releases are deployed by GitHub Actions over SSH as a restricted `deploy` user that may run exactly one command, `/opt/mobileshop/deploy.sh <tag>` .

Caddy adds HSTS, `nosniff` , frame and referrer policies, allows the camera only for this site, and limits uploads to 10 MB. With `APP_ENV=prod` the backend refuses a weak `SECRET_KEY` and hides `/docs` .

Step 1: prepare a fresh server

On a new Ubuntu or Debian VPS, as root:

1. Update the system: `apt update && apt upgrade -y`
2. Install the basic tools: `apt install -y git curl openssl cron`
3. Install Docker Engine with the Compose plugin: `curl -fsSL https://get.docker.com | sh`
4. Check: `docker compose version`
5. Create the deploy user if the server does not already have one: `adduser --disabled-password --gecos "" deploy`
6. Create its SSH folder: `install -d -m 700 -o deploy -g deploy /home/deploy/.ssh`

Step 2: make sure Traefik is there

The compose file expects what the owner's server already has. On a fresh server you must provide it:

- a Docker network named `n8n_default` (create it with `docker network create n8n_default` if it does not exist);
- a Traefik container on that network, publishing ports 80 and 443, with the Docker provider switched on, an entrypoint named `web` on port 80, an entrypoint named `websecure` on port 443, and a Let's Encrypt certificate resolver named `mytlschallenge` using the TLS challenge.

If you run a different proxy, or Traefik with other names, change the network name and the `traefik.` labels of the `web` service in `deploy/docker-compose.yml` to match, in the repository (the server copy is replaced on each deploy).

> **Note:** `setup-vps.sh` stops with an error if the `deploy` user or the `n8n_default` network is missing.

Step 3: the domain

1. Choose the address, for example `shop.<your-domain>`.
2. At your DNS provider, add an A record for it pointing to the server's public IP address.
3. Wait until `ping <your-app-domain>` answers from the server's address.

Step 4: install the stack files

As root on the server:

1. Get the repository: `git clone --depth 1 https://github.com/AbdurRehmanAfzal/mobile-shop-software.git /tmp/ms`
2. Run the one-time setup: `bash /tmp/ms/deploy/setup-vps.sh`

The script creates `/opt/mobileshop` with `docker-compose.yml`, `deploy.sh` and `backup.sh`; creates `/opt/mobileshop/.env` (mode 600) from `deploy/.env.example` with a fresh `POSTGRES_PASSWORD` and `SECRET_KEY`; adds the sudoers rule `/etc/sudoers.d/mobileshop-deploy` that lets `deploy` run only `/opt/mobileshop/deploy.sh`; and installs a cron job at 02:30 for the nightly backup.

Step 5: fill in the server .env

1. Open it: `nano /opt/mobileshop/.env`
2. Set `APP_DOMAIN` to the address from step 3.
3. Set `SUPERUSER_PHONES` to the product owner's login number.
4. If you use a fork, set `IMAGE_PREFIX` to `ghcr.io/<your-github-user-in-lowercase>/mobile-shop`.
5. Leave the optional keys (WhatsApp, SMTP, AI, rclone) empty for now; chapter 5 explains each one.

> **Important:** `/opt/mobileshop/.env` holds the database password and the token secret. Never commit it, paste it into chat or copy it into documentation.

Step 6: access to the images

The images are on GitHub's registry. If the packages are private, log in on the server once with a GitHub personal access token that has `read:packages` : `docker login ghcr.io -u <github-user>`

Step 7: connect GitHub Actions

1. On your own PC, make a key pair for the deploy job: `ssh-keygen -t ed25519 -f ~/.ssh/mobileshop_deploy -C deploy@github`
2. Add the public half (`mobileshop_deploy.pub`) to `/home/deploy/.ssh/authorized_keys` on the server, owned by `deploy` with mode 600.
3. In the GitHub repository, add the secrets `VPS_HOST` (the server's address), `VPS_USER` (`deploy`) and `VPS_SSH_KEY` (the whole private key file).
4. Add the repository variable `APP_URL` = `https://<your-app-domain>` .
5. Add the Android signing secrets (chapter 12) if you want a release-signed APK.

> **Note:** if you deploy a fork, also change the repository address in `deploy/deploy.sh` (the `RAW=` line). Each deploy downloads `docker-compose.yml` , `backup.sh` and `deploy.sh` of that exact commit from the repository, so changes to them reach the server.

Step 8: the first release

1. Merge `develop` into `main` and push, or run Actions → Deploy → Run workflow.
2. On the server, follow the backend log: `docker compose -f /opt/mobileshop/docker-compose.yml logs -f backend`
3. `deploy.sh` pulls the images (retrying up to four times), starts the stack, waits for `/health` and writes the tag to `/opt/mobileshop/.current-tag` . If the new version is not healthy it prints the backend log and goes back to the previous tag.
4. The first request on a new domain makes Traefik fetch the certificate; for a minute HTTPS may fail. The workflow's final check retries for two minutes.

Step 9: the first login and the keys

Public signup is off in production, so the first account is made on the server. The command asks for the shop, name, number and a password that is not echoed:

1. `cd /opt/mobileshop`
2. `docker compose --env-file .env exec backend python -m app.cli create-owner`
3. Use the number that is in `SUPERUSER_PHONES` , then open `https://<your-app-domain>` , log in, and open customer shops from More → Super admin.

Make the production key pair for license and renewal keys once:

1. `docker compose --env-file .env exec backend python -m app.modules.plan.renewal keygen`
2. Paste both printed lines (`RENEWAL_PRIVATE_KEY=` and `RENEWAL_PUBLIC_KEY=`) into `/opt/mobileshop/.env` .
3. Restart the backend: `docker compose --env-file .env up -d backend`

Keep the private key only on this server. The public key goes into every install, including the offline Windows build. Then, in the app as super admin, replace the billing placeholders under More → Super admin → Fees → Billing settings (bank, wallets, support numbers); the checklist is `docs/21-OWNER-SETUP-CHECKLIST.md` .

Step 10: optional services

- **WhatsApp:** in Meta Business, add a business number, note its Phone number ID and make a permanent system-user token with `whatsapp_business_messaging` . Put `WHATSAPP_TOKEN` and `WHATSAPP_PHONE_ID` in `.env` . Messages the shop starts (reminders, receipts, the night report) need approved templates, mapped in `WHATSAPP_TEMPLATES` . Restart the backend.
- **Email:** set the `SMTP_` keys and restart the backend.
- **AI:** set `ANTHROPIC_API_KEY` (and optionally `AI_DAILY_LIMIT`) and restart the backend.
- **Old stock without barcodes** (only on a server that has data from before barcodes existed): `docker compose --env-file .env exec backend python -m app.modules.barcodes.backfill`

After every `.env` change: `docker compose --env-file .env up -d backend`

Backups

The cron job runs `/opt/mobileshop/backup.sh` at 02:30 and logs to `/var/log/mobileshop-backup.log` . It writes a `pg_dump` (custom format) and a tarball of the uploads to `/opt/mobileshop-backups` and keeps 14 days there. With `RCLONE_REMOTE` set in `.env` and rclone installed, it also copies the new files off the server and deletes copies older than 30 days there.

To set up an encrypted Google Drive copy, as root:

1. Install rclone: `curl https://rclone.org/install.sh | bash`
2. Run `rclone config` , add a remote named `gdrive` of type `drive` , scope full, and answer "n" to the automatic browser login; run the printed `rclone authorize` command on your own PC and paste the token back.
3. Run `rclone config` again and add a remote named `gdrive-safe` of type `crypt` over `gdrive:mobileshop-backups` , with standard filename encryption and two strong passwords. Keep both passwords safe; without them the backups cannot be opened. The backups contain CNIC photos, so never skip the encryption.
4. Switch it on: `echo 'RCLONE_REMOTE=gdrive-safe:' >> /opt/mobileshop/.env`
5. Run it once: `/opt/mobileshop/backup.sh` (the last line says "offsite copy ok").
6. Check: `rclone ls gdrive-safe: | tail`

To restore a database dump:

1. `cd /opt/mobileshop`
2. `docker compose exec -T db pg_restore -U mobileshop -d mobileshop --clean < /opt/mobileshop-backups/db-YYYYMMDD-HHMM.dump`

Uploads are restored by unpacking the matching `uploads-YYYYMMDD-HHMM.tgz` into the `mobileshop_uploads` volume.

Day to day on the server

Task	Command (in /opt/mobileshop)
Status	`docker compose ps`
Backend log	`docker compose logs --tail 100 backend`
Web log	`docker compose logs --tail 100 web`
Version running	`cat .current-tag`
Database version	`docker compose --env-file .env exec backend alembic current`
Roll back to an older release	`sudo /opt/mobileshop/deploy.sh <older-commit-sha>`
Restart the backend after an .env change	`docker compose --env-file .env up -d backend`

> **Important:** a rollback does not undo migrations. Rolling back across a schema change needs a database restore. Do not touch the production server without the owner's go-ahead.

Running the production stack without CI

To try the production images on any machine with Docker (this is what CI's stack job does), from a checkout of the repository:

1. Create Traefik's network: `docker network create n8n_default`
2. Go to the deploy folder: `cd deploy`
3. In Git Bash or Linux, start the stack with test values: `APP_DOMAIN=ci.localhost POSTGRES_PASSWORD=<local-only> SECRET_KEY=<local-only-at-least-32-characters> PUBLIC_SIGNUP=true IMAGE_PREFIX=mobileshop-ci IMAGE_TAG=local docker compose -f docker-compose.yml -f ci/compose.ci.yml up -d --build --wait db backend web`
4. The app is at `http://localhost:8080`. Run the smoke test from the repository root: `bash deploy/ci/smoke.sh`

12. The Android app and the Windows PC version

The installable web app (PWA)

The web app is a PWA (`vite-plugin-pwa`). The service worker caches only the app shell, never API data or uploads, which are per shop and sensitive. On Android Chrome, open the site and choose Install app. A full install and the offline shell need HTTPS.

The Android app

The Android app in `frontend/android/` is a Capacitor 6 shell (app id `pk.mobileshop.app`) around the same React build, which talks to the cloud API. `npm run build:native` runs `tsc -b` , then `vite build --mode native` (no service worker, so an updated app never shows an older cached version) and `cap sync android` . The server must allow the origins `https://localhost` and `capacitor://localhost` in `CORS_ORIGINS` (the compose file does this).

Build it in GitHub Actions (no Android Studio needed)

`android.yml` runs inside every release (called by `deploy.yml`) or by hand from Actions → Android → Run workflow:

1. Version code = minutes since 1970, so every build installs over the previous one; version name = the date plus the short commit.
2. Node 22 and Temurin JDK 17; `npm ci` ; `npm run build:native` with `VITE_API_URL` = the repository variable `APP_URL` (the job fails if it is not set).
3. With the signing secrets, `gradlew assembleRelease` ; without them, `gradlew assembleDebug` and a warning (a test APK only).
4. The APK is uploaded as the artifact `mobile-shop-android-<date>-<commit>` , kept 90 days. In a release, the deploy job copies it into the web image, so the latest app is always at `https://<your-app-domain>/app.apk` ; the super-admin page shows that link with Download, WhatsApp and Share buttons.

The release key (one time, keep it forever)

Every update must be signed with the same key, or phones refuse to install it over the old app. Make it once, on any machine with Docker:

1. `mkdir -p /root/android-key`
2. `docker run --rm -it -v /root/android-key:/k eclipse-temurin:17-jdk keytool -genkeypair -v -keystore /k/mobileshop-release.jks -alias mobileshop -keyalg RSA -keysize 2048 -validity 10000 -dname "CN=<your company>, O=<your company>, C=PK"`
3. Print it as one base64 line: `base64 -w0 /root/android-key/mobileshop-release.jks; echo`
4. In GitHub secrets: `ANDROID_KEYSTORE_BASE64` (that line), `ANDROID_KEYSTORE_PASSWORD` (the password you typed), `ANDROID_KEY_ALIAS`

(`mobileshop`).

5. Keep a copy of the `.jks` file and its password off the server, for example in a password manager. Losing it means the app can never be updated in place.

`frontend/android/app/build.gradle` reads `APP_VERSION_CODE` , `APP_VERSION_NAME` , `ANDROID_KEYSTORE_FILE` , `ANDROID_KEYSTORE_PASSWORD` , `ANDROID_KEY_ALIAS` and, optionally, `ANDROID_KEY_PASSWORD` (falls back to the keystore password) from the environment.

Build it on your own PC

1. Install Android Studio (it brings JDK 17 and the Android SDK).
2. In `frontend/` , set the API address for this build. PowerShell: `$env:VITE_API_URL = "https://<your-app-domain>"`
3. Build and sync: `npm run build:native`
4. Open the project: `npx cap open android`
5. In Android Studio, Build → Generate Signed App Bundle or APK, with the release key.
6. Test the APK on a phone; for the Play Store, upload the `.aab` to an internal testing track first.

To install an APK by hand, copy it to the phone (or download it from `/app.apk`), open it and allow "Install unknown apps" once. App icons and the splash screen come from `frontend/assets/` and are generated with `npx @capacitor/assets generate --android` .

In the app, printing and downloads open in Chrome through a two-minute link for that one address (`POST /api/v1/auth/link` , with `autoprint=1` for print pages), because the WebView has no print dialog or download manager. iOS would use the same Capacitor project but needs a Mac with Xcode and an Apple developer account; it is not built yet.

The Windows PC version (one shop, no internet)

For shops without reliable internet, the same backend runs on the shop's PC with SQLite and serves the same web app (ADR-029, `docs/15-LOCAL-WINDOWS.md`). One shop per install; there is no cloud sync.

What the shop gets:

- `MobileShop-Setup-<version>.exe` installs to `Program Files\Mobile Shop` , with Start Menu and Desktop shortcuts and an optional "start with Windows".
- `MobileShop.exe` puts an icon in the tray and opens the browser at `http://127.0.0.1:8787` . The tray menu has Open Mobile Shop, Backup now, Open data folder and Quit.
- The data lives in `%LOCALAPPDATA%\MobileShop` : `mobileshop.db` , `uploads\` , `backups\` , `settings.env` and `mobileshop.log` . Updates and uninstalls never touch it.
- On first run the shop creates its own account (public signup is on for this one PC). `LOCAL_MODE=true` makes its organisation plan `chain` with no expiry.
- Backup now writes `backups\mobileshop-<date>.db` and a copy on the Desktop. To restore: quit, copy a backup over `mobileshop.db` , start again.

- Phones on the shop's Wi-Fi can use it after adding `MOBILESHOP_HOST=0.0.0.0` to `settings.env` and allowing port 8787 in Windows Firewall; then open `http://<PC-IP>:8787`.

How it is built:

- `backend/app/local.py`: the launcher (data folder, `settings.env`, Alembic `upgrade head` with the bundled migrations, uvicorn in a thread, the browser, the pystray tray with a console fallback).
- `WEB_DIR`: the backend mounts the built web app (`/assets` as static files, every other non-API path falls back to `index.html`).
- `local/mobileshop.spec` (PyInstaller, one folder), `local/installer.iss` (Inno Setup 6), `local/build.ps1`.
- The Python extra `backend[local]`: pyinstaller, pystray, pillow, tzdata.

Build the Windows version

On a Windows PC with Python 3.12, Node 22 and, for the installer, Inno Setup 6 (`iscc` on PATH), from the repository root in PowerShell:

1. Build: `.\local\build.ps1 -Version 2.1.0`
2. The script builds the web app with an empty `VITE_API_URL` (same origin), makes `backend\.venv-local` with the `local` extra, runs PyInstaller into `dist\MobileShop\`, and with Inno Setup writes `dist\MobileShop-Setup-2.1.0.exe`.
3. Without Inno Setup you still get `dist\MobileShop\` to zip.

Or in GitHub: Actions → Windows → Run workflow with a version. The workflow builds on a Windows runner, smoke-tests the bundle (`/health`, the app page, `/api/v1/public/info` on port 8790 with a temporary data folder), runs Inno Setup and uploads the artifact `MobileShop-Setup`.

To make offline installs accept license and renewal keys from your SaaS server, the build must carry your production `RENEWAL_PUBLIC_KEY` (add it to the shop's `settings.env`, or change the default in `backend/app/core/config.py` before building). To let an offline install report its heartbeat when it has internet, add `SAAS_URL=https://<your-app-domain>` to its `settings.env`.

13. Troubleshooting

Symptom	Cause and fix
`UnicodeEncodeError: 'charmap' codec can't encode`	`PYTHONUTF8=1` is not set in this terminal.
`py -3.12` not found, or the wrong Python in the venv	Install Python 3.12; delete `backend/.venv` and recreate it with `py -3.12 -m venv .venv`.
Signup says it is closed (`signup_closed` , 403) on your PC	Add `PUBLIC_SIGNUP=true` to `backend/.env` and restart the backend.
No Super admin under More	Your login number is not in `SUPERUSER_PHONES` ; add it and restart the backend (on the server: `up -d backend`).
A route you just wrote returns 404	The Windows reloader missed it, or an orphan process holds port 8000. Find it with `netstat -ano` and the port, stop it with `taskkill /PID <pid> /F` , and start uvicorn again with `--reload-dir app` . Also check the router is in `main.py` and fixed paths come before `/{id}` .
`no such column` or `table has no column`	Run `alembic upgrade head` . If it still fails, a migration is missing; `tests/test_migrations.py` will fail too.
A downgrade fails on PostgreSQL but works on SQLite	A constraint name differs between `upgrade()` and `downgrade()` .
`shop_required` or `shop_forbidden` errors	The user has several shops and the `X-Shop-Id` header is missing, or the id is not one of theirs.
The app shows raw keys such as `expense.billRef`	The key is missing in that locale file, or a namespace object was overwritten with a string.
English words in Urdu mode	A hard-coded string or a missing `ur.json` key; `e2e/urdu.spec.ts` finds them.
The phone cannot open `http://<PC-IP>:5173`	Same Wi-Fi? Started with `npm run dev:lan` , not `npm run dev` ? Node allowed in Windows Firewall on the Private profile?
The camera opens the camera app and the form resets	Use `npm run dev:phone` (https on 5174); long forms must use `useDraft` .
`admin.spec.ts` or another super-admin spec fails	Check `03990000099` is in `SUPERUSER_PHONES` ; if it fails only in a full run, run it alone (known flake).

Symptom	Cause and fix
The backend container will not start in production	`docker compose logs --tail 200 backend`. A weak or missing `SECRET_KEY` stops it on purpose; a migration error shows here too.
`set POSTGRES_PASSWORD in .env` or `set APP_DOMAIN in .env` from Compose	The server `.env` is missing that key, or Compose was run without `--env-file .env` from `/opt/mobileshop`.
A setting in `/opt/mobileshop/.env` has no effect	The compose file does not pass that variable to the backend (chapter 5), or the backend was not restarted with `up -d backend`.
The deploy job fails with "VPS_SSH_KEY is not a valid private key"	The secret must hold the whole private key file, including its first and last lines.
The deploy rolled back by itself	The new version failed `/health`. Read the backend log that `deploy.sh` printed (often a migration error).
`could not pull images`	The server cannot reach `ghcr.io`, or the packages are private and the server is not logged in (`docker login ghcr.io`).
The final check fails with curl exit code 60 right after the first deploy	Traefik is still fetching the certificate for a new domain; wait a minute and check `/health` again.
`/app.apk` returns 404	The Android job failed in that release (often the `APP_URL` variable is missing); the site was deployed anyway.
The APK will not install over the old one	It is signed with a different key (debug against release) or has a lower version code; uninstall the old app or use the release key.
The Android app cannot log in	`APP_URL` was wrong at build time, or the server's `CORS_ORIGINS` lacks `https://localhost` and `capacitor://localhost`.
WhatsApp messages stay in Messages as "Failed"	The row shows the reason. Outside the 24-hour window Meta needs an approved template mapped in `WHATSAPP_TEMPLATES`.
The offline Windows version does not open	Look at `mobileshop.log` in `%LOCALAPPDATA%\MobileShop`; check nothing else uses port 8787.
Phones cannot reach the offline Windows version	Add `MOBILESHOP_HOST=0.0.0.0` to `settings.env`, restart the program and allow port 8787 in Windows Firewall.

Tip: when something is unclear, the newest truth is in the code and in docs/06-PROGRESS.md and the last entry of docs/09-SESSION-LOG.md . If this manual disagrees with CLAUDE.md , CLAUDE.md wins, and this manual should be fixed.